Задача 1

1. Анализ на задачата

Какво се изисква: Трябва да дефинираме структура 'Medicine' с пет полета (име, дата на годност, код, цена, количество). След това трябва да прочетем данни от предварително създаден бинарен файл 'medicines.bin' и да запишем тези данни в динамичен масив (масив, чийто размер се определя по време на изпълнение на програмата). Трябва да хващаме грешки, ако файлът липсва.

Какви входни/изходни данни има: Вход е файлът 'medicines.bin'. Изходът е указател към създадения динамичен масив в паметта и броят на прочетените елементи.

2. Логика (начинаещо ниво)

Стъпка 1: Създаваме структурата с точните типове. Кодът е 13-цифрен, затова 'unsigned long long' е най-подходящ. Датата е низ (масив от символи) с размер 8 (7 символа за ММ.ГГГГ + 1 за терминиращата нула '\0').

Стъпка 2: За да заделим динамичен масив, първо трябва да разберем колко лекарства има във файла. Отваряме файла за четене в бинарен режим ('rb').

Стъпка 3: Местим 'курсора' за четене в края на файла, взимаме неговия размер в байтове и делим на размера на една структура. Така получаваме броя на лекарствата.

Стъпка 4: Използваме 'malloc' за да заделим памет за този брой елементи.

Стъпка 5: Връщаме курсора в началото, прочитаме всичко наведнъж с 'fread' и затваряме файла.

3. С код (стандартен С)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Medicine {
    char name[30];
    char expiry_date[8];
    unsigned long long barcode;
    float price;
    int quantity;
};

struct Medicine* load_medicines(int* count) {
    FILE* file = fopen("medicines.bin", "rb");
    if (file == NULL) {
        printf("Greshka pri otvarqne na faila!\n");
        *count = 0;
        return NULL;
```

```
    }

    fseek(file, 0, SEEK_END);
    long file_bytes = ftell(file);
    fseek(file, 0, SEEK_SET);

    *count = file_bytes / sizeof(struct Medicine);

    if (*count == 0) {
        fclose(file);
        return NULL;
    }

    struct Medicine* arr = (struct Medicine*)malloc((*count) * sizeof(struct
Medicine));
    if (arr == NULL) {
        printf("Greshka pri zadelqne na pamet!\n");
        fclose(file);
        return NULL;
    }

    fread(arr, sizeof(struct Medicine), *count, file);
    fclose(file);

    return arr;
}
```

4. Обяснение на кода ред по ред

- struct Medicine { ... }: Дефинира потребителски тип данни, събиращ разнородни стойности.

- char expiry_date[8]: Масив от символи за датата (7 символа за текста + 1 за края на низа).

- unsigned long long barcode: Най-големият целочислен тип, тъй като 13 цифри не се събират в стандартен int (макс ~2 милиарда - 10 цифри).

- struct Medicine* load_medicines(int* count): Функция, която връща указател към масива. Приема 'count' по адрес, за да може да запише и върне броя на прочетените елементи в основната програма.

- FILE* file = fopen(...): Отваря файла в режим 'rb' (read binary - четене на бинарен файл).

- fseek(file, 0, SEEK_END): Премества позицията за четене в самия край на файла.

- ftell(file): Връща текущата позиция в байтове (на практика - големината на целия файл).

- fseek(file, 0, SEEK_SET): Връща позицията за четене обратно в началото.

- *count = file_bytes / sizeof(struct Medicine): Изчислява броя на записите.

- malloc((*count) * sizeof(...)): Резервира блок памет (в хийпа), достатъчен да побере всички записи.

- fread(arr, sizeof(struct Medicine), *count, file): Извлича наведнъж всички структури от файла и ги пълни в масива 'arr'.

5. Стъпково изпълнение

1. Програмата се опитва да отвори 'medicines.bin'. Ако не съществува, отпечатва грешка и връща NULL.

2. Ако съществува, 'превърта' до края, за да го премери. Например, ако файлът е 1000 байта, а структурата е 50 байта, пресмята 1000/50 = 20 лекарства.

3. Заделя памет за точно 20 лекарства.

4. Прочита тези 20 лекарства с едно действие и ги поставя в паметта.

5. Затваря файла и връща готовия масив на потребителя.

6. Чести грешки

- Използване на int за 13-цифрен код (води до препълване).

- Забравяне на режима 'rb' и отваряне само с 'r' (може да доведе до счупване на данните при Windows).

- Забравяне на '\0' при определяне на размера на низа за дата (оставят 7 вместо 8).

- Забравяне на връщането на курсора в началото (SEEK_SET) след измерването.

Задача 2

1. Анализ на задачата

Какво се изисква: Функция, която приема масив, брой елементи и дата. Тя трябва да намери лекарствата, изтичащи ПРЕДИ тази дата, и да ги върне (вероятно като нов динамичен масив).

Какви входни/изходни данни има: Вход - указател към основния масив, целочислен размер и низ за целевата дата. Изход - нов динамичен масив (и нов размер чрез указател) или NULL, ако няма такива.

2. Логика (начинаещо ниво)

Стъпка 1: Тъй като датата е низ ('10.2024'), директното сравняване с strcmp няма да работи коректно (защото '12.2023' лексикографски е след '10.2024', въпреки че като дата е преди нея).

Стъпка 2: Трябва да извлечем месеца и годината като числа чрез sscanf.

Стъпка 3: Обхождаме целия масив с един цикъл (от 0 до size). За всяко лекарство вадим неговия месец и година.

Стъпка 4: Сравняваме: ако годината на лекарството е по-малка от търсената ИЛИ (ако годините са равни, но месецът е по-малък), това лекарство отговаря на условието.

Стъпка 5: Ако отговаря, заделяме/разширяваме нов масив и го добавяме вътре.

3. С код (стандартен С)

```
struct Medicine* get_expired_before(struct Medicine* arr, int size, const char*
target_date, int* new_size) {
    int target_m, target_y;
    sscanf(target_date, "%d.%d", &target_m, &target_y);

    struct Medicine* result = NULL;
    int count = 0;

    for (int i = 0; i < size; i++) {
        int m, y;
        sscanf(arr[i].expiry_date, "%d.%d", &m, &y);

        if (y < target_y || (y == target_y && m < target_m)) {
            count++;
            struct Medicine* temp = (struct Medicine*)realloc(result, count *
sizeof(struct Medicine));
            if (temp != NULL) {
                result = temp;
                result[count - 1] = arr[i];
            } else {
                printf("Greshka pri realloc!\n");
                free(result);
                *new_size = 0;
                return NULL;
            }
        }
    }

    *new_size = count;
    return result;
}
```

4. Обяснение на кода ред по ред

- sscanf(target_date, "%d.%d", &target_m, &target_y): Чете низа и автоматично извлича числата преди и след точката.

- struct Medicine* result = NULL: Подготвяме празен указател за новия масив с резултати.

- for (int i = 0; i < size; i++): Стартираме цикъл, който преминава през всяко лекарство.

- if (y < target_y || (y == target_y && m < target_m)): Основната логика за сравнение на дати. Първо гледаме годината. Ако е същата, гледаме месеца.

- realloc(result, count * sizeof(...)): Разширява текущо заделената памет с още едно място. Ако 'result' е NULL (при първото намиране), realloc работи като malloc.

- result[count - 1] = arr[i]: Копира цялата структура от стария масив в новото свободно място (в края) на масива-резултат.

5. Стъпково изпълнение

1. Разделяме търсената дата (напр. '06.2024') на m=6 и y=2024.

2. Взимаме първото лекарство. Неговата дата е '10.2023'. y=2023, m=10. Тъй като 2023 < 2024, то изтича преди търсената дата.

3. Заделяме памет за 1 елемент и го слагаме вътре. Броячът става 1.

4. Взимаме второ лекарство с '08.2024'. y=2024 (не е по-малко), сравняваме месеците: 8 < 6 не е вярно. Пропускаме го.

5. Връщаме новия масив и неговия размер.

6. Чести грешки

- Използване на strcmp за дати във формат ММ.ГГГГ.

- Използване на malloc в цикъла вместо realloc (води до презаписване на стария указател и изтичане на памет).

- Грешно формулирано логическо условие за сравнение на година и месец.

Задача 3

1. Анализ на задачата

Какво се изисква: Функция, която приема масива и два реални параметъра (min и max цена). Трябва да филтрира лекарствата и да ги запише в ТЕКСТОВ файл 'offer.txt' в точно определен формат.

Какви входни/изходни данни има: Вход - масивът, размерът, минимална и максимална цена. Изход - създаден файл 'offer.txt' и върнат брой на записаните елементи като цяло число (или 0).

2. Логика (начинаещо ниво)

Стъпка 1: Отваряме 'offer.txt' в режим за писане 'w'. Ако файлът съществува, ще бъде презаписан. Ако не - създаден.

Стъпка 2: Правим променлива-брояч, започваща от 0.

Стъпка 3: Обхождаме всички елементи на масива с цикъл.

Стъпка 4: Проверяваме дали цената е в диапазона: цена >= min_price И цена <= max_price.

Стъпка 5: Ако да, записваме данните ред по ред с fprintf в указания формат и увеличаваме брояча.

Стъпка 6: Затваряме файла и връщаме брояча.

3. С код (стандартен С)

```
int write_offers(struct Medicine* arr, int size, float min_price, float
max_price) {
    FILE* file = fopen("offer.txt", "w");
    if (file == NULL) {
        printf("Greshka pri suzdavane na faila!\n");
        return 0;
    }

    int count = 0;
    for (int i = 0; i < size; i++) {
        if (arr[i].price >= min_price && arr[i].price <= max_price) {
            fprintf(file, "%s\n", arr[i].name);
            fprintf(file, "%s\n", arr[i].expiry_date);
            fprintf(file, "%llu\n", arr[i].barcode);
            fprintf(file, "%.2fleva\n", arr[i].price);
            count++;
        }
    }

    fclose(file);
    return count;
}
```

4. Обяснение на кода ред по ред

- FILE* file = fopen("offer.txt", "w"): 'w' означава write text (писане на текстов файл).

- if (arr[i].price >= min_price && arr[i].price <= max_price): Операторът && (ЛОГИЧЕСКО И) гарантира, че и двете граници са спазени.

- fprintf(file, "%s\n", arr[i].name): Записва стринга (името) във файла и добавя нов ред (\n).

- fprintf(file, "%llu\n", arr[i].barcode): %llu е форматът за unsigned long long (13-цифрения баркод).

- fprintf(file, "%.2fleva\n", arr[i].price): %.2f закръгля реалното число до 2 знака след запетаята. Думата 'leva' е залепена веднага след формата, както се иска в условието.

- return count: Връща броя на реално записаните стоки.

5. Стъпково изпълнение

1. Програмата създава празен текстови файл offer.txt.

2. Започва проверка на всяко лекарство. Ако търсим между 5.00 и 10.00 лева, и срещнем Аналгин за 6.90.

3. Условието 6.90 >= 5.00 и 6.90 <= 10.00 е ИСТИНА.

4. Отпечатват се 4 реда във файла за Аналгина.

5. Броячът става 1. Продължава със следващите.

6. Накрая връща числото 1 (ако е само Аналгинът).

6. Чести грешки

- Използване на '%d' за баркода - ще принтира боклук заради типа unsigned long long.

- Забравяне на '\n' при fprintf, което води до сляпване на целия текст на един ред.

- Отваряне в режим 'wb' (бинарен) вместо 'w' (текстов).

Задача 4

1. Анализ на задачата

Какво се изисква: Функция, която премахва (изтрива) елемент от съществуващия масив по зададено име и дата. Ако не е намерен, принтира съобщение.

Какви входни/изходни данни има: Тъй като масивът се променя (както данните вътре, така и неговият реален размер в паметта), трябва да подадем масива и размера по адрес (като указател към указател и указател към int).

2. Логика (начинаещо ниво)

Стъпка 1: Търсим индекса на елемента, който съвпада по име и дата (с използване на strcmp за низове).

Стъпка 2: Ако такъв индекс не бъде открит, отпечатваме съобщение и прекратяваме функцията.

Стъпка 3: Ако е открит, трябва да 'изтрием' елемента. Това става като изместим всички елементи вдясно от него с една позиция наляво, запълвайки дупката.

Стъпка 4: Намаляваме променливата за размер с 1.

Стъпка 5: Използваме realloc, за да свием физически заделената памет на масива с един елемент надолу.

3. С код (стандартен С)

```
void delete_medicine(struct Medicine** arr, int* size, const char* t_name,
const char* t_date) {
    int found_index = -1;

    for (int i = 0; i < *size; i++) {
        if (strcmp((*arr)[i].name, t_name) == 0 &&
strcmp((*arr)[i].expiry_date, t_date) == 0) {
            found_index = i;
            break;
        }
    }

    if (found_index == -1) {
        printf("Lekarstvoto ne e namereno.\n");
        return;
    }

    for (int i = found_index; i < *size - 1; i++) {
        (*arr)[i] = (*arr)[i + 1];
    }

    (*size)--;

    if (*size > 0) {
        struct Medicine* temp = (struct Medicine*)realloc(*arr, (*size) *
sizeof(struct Medicine));
        if (temp != NULL) {
            *arr = temp;
        }
    } else {
        free(*arr);
        *arr = NULL;
    }

    printf("Lekarstvoto e iztrito uspeshno.\n");
}
```

4. Обяснение на кода ред по ред

- struct Medicine** arr: Използваме двоен указател. Така можем да променим самия указател на масива в главната функция (main), ако realloc го премести на друг адрес в паметта.

- strcmp((*arr)[i].name, t_name) == 0: Функцията strcmp сравнява низове. Ако върне 0, низовете са напълно еднакви.

- found_index = i; break;: Запомняме къде сме намерили елемента и веднага спираме цикъла.

- for (int i = found_index; i < *size - 1; i++): Този цикъл започва от намерената позиция и 'дърпа' всеки следващ елемент една позиция назад. (*arr)[i] = (*arr)[i + 1] презаписва текущия с този отдясно.

- (*size)--: Намаляваме логическия размер.

- if (*size > 0) { realloc... }: Свиваме физическия размер с realloc. Ако масивът остане празен (размер 0), просто го освобождаваме с free().

5. Стъпково изпълнение

1. Имаме масив с 3 елемента (A, B, C). Искаме да изтрием B.

2. Цикълът намира B на индекс 1.

3. Вторият цикъл измества: на позиция 1 записва C. Сега масивът е (A, C, C).

4. Размерът се намалява от 3 на 2.

5. Realloc отрязва последния елемент. Масивът остава (A, C) с размер 2.

6. Чести грешки

- Сравняване на низове с '==' (напр. arr[i].name == t_name). В C това сравнява адресите на паметта, не съдържанието.

- Забравяне за прехвърляне на масива по адрес (чрез двойния указател). Ако подадем само 'struct Medicine* arr', всяко релокиране (промяна на адреса) ще се загуби при връщане в main().

- Излизане извън границите при изместването (цикъл до *size вместо до *size - 1).

Основни концепции за изпита по C

Променливи

Контейнери за данни с определен тип (int, float, char). Важно е да съобразявате обхвата на типа (напр. 13-цифрен баркод изисква unsigned long long, не int).

Цикли

Използват се за повтаряеми действия. 'for' се ползва най-често при масиви, когато знаем точния брой елементи (size). 'while' е удобен при четене от файлове до достигане на край (EOF).

Условия

if, else if, else управляват логическия поток. Важно: използвайте && за И, || за ИЛИ. Запомнете разликата между '=' (присвояване) и '==' (сравнение).

Масиви (статични и динамични)

Статичните имат фиксиран размер при компилация (напр. char name[30]). Динамичните (като масива от структури тук) се заделят по време на изпълнение с malloc/calloc и могат да се преоразмеряват с realloc.

Функции

Парчета код за многократна употреба. Когато искате функция да промени стойността на дадена променлива отвън (напр. размера на масив), трябва да я подадете 'по адрес' чрез указател (int* size).

Често срещани шаблони

- if/else конструкции: Използват се при валидация на вход или отваряне на файл (if(file == NULL) return;).

- for/while цикли: Стандартен шаблон за обхождане 'for(int i=0; i<size; i++)'.

- Работа с масиви: Добавяне в динамичен масив обикновено следва стъпките: count++, realloc(count*size), масив[count-1] = нов елемент.

- Функции с параметри: Ако структурата е голяма, подаването ѝ е като масив или указател, за да не се копира излишно.

Стратегия за решаване на изпит

1. Как да се анализира задача: Първо извадете типовете данни (колко байта/цифри). Намерете входа и изхода.

2. Как да се избира структура: Използвайте typedef struct или struct. Дръжте нещата прости, избягвайте сложни вложени структури, освен ако не се изискват.

3. Как да се проверява решение: Мислете за 'гранични случаи' (corner cases) - какво ако файлът липсва? Какво ако масивът е празен? Какво ако нито един елемент не отговаря на условието?